



Quorum — Complete Engineering Specification & System Documentation

Autonomous Multi-Agent AI Research Swarms, Topological DAG Execution, Academic DOI Fact-Checking, Local Offline Ollama Engine & Model Context Protocol

Lead Engineer & Architect:	V.A. SAI VENKATESH
Multi-Agent Architecture:	Topological DAG Orchestrator, Async ReAct Swarm, Semantic Scholar & CrossRef Fact-Checker
Dual AI Inference Engine:	Cloud Providers (Gemini 1.5, GPT-4o, Claude 3.5) + Offline Air-Gapped Ollama (Llama 3, Mistral)
Full-Stack Implementation:	FastAPI 0.115+, SQLAlchemy 2.0 Async, PostgreSQL pgvector, Next.js 14 SSR, Tailwind CSS
Document Classification:	Enterprise Engineering Specification v1.0 (September 2026)

★ LEAD ARCHITECT SYSTEM VISION — V.A. Sai Venkatesh (Lead Architect)

"Current generative AI tools provide persuasive, fluent prose but frequently hallucinate citations, invent non-existent DOIs, and collapse under multi-step research investigations. Quorum was engineered to eradicate algorithmic hallucination in mission-critical research. By orchestrating a topological DAG of specialized agents—planners, academic retrievers, peer-review fact-checkers, and codebase synthesizers—Quorum guarantees mathematical attribution, verifiable source grounding, and complete transparency from raw query to final paper."

Executive Overview: The Autonomous Research & Verification Imperative

Modern scientific research, competitive intelligence, and software engineering demand synthesis of massive, disparate literature. However, monolithic language models suffer from context decay, citation fabrication, and inability to interact with live software codebases. **Quorum introduces an autonomous, verified multi-agent paradigm** where research tasks are decomposed into dependency-governed DAGs.

Whether operating across global academic databases or inside secure, air-gapped corporate environments using local Ollama models, Quorum synthesizes comprehensive technical documentation, research papers, and codebase audits with strict DOI verification and live AST inspection.

System Architecture & Computer Science Foundations

Quorum is structured across four decoupled architectural layers with strict asynchronous protocol boundaries: Presentation (Next.js SSR), Distributed Coordination (FastAPI & Arq Workers), Execution (DAG Swarms), and Verified Storage (PostgreSQL & pgvector).

Computer Science Pattern	Implementation & File Location	Engineering Rationale
Topological DAG Swarm	apps/api/src/agents/orchestrator.py	Enforces strict dependency order across parallel research tasks. Eliminates race conditions and guarantees synthesis only occurs after fact-check completion.
Polymorphic Provider Strategy	apps/api/src/agents/providers.py	Abstracts LLM inference across Gemini, OpenAI, Claude, and local Ollama. Switches models transparently with zero agent code changes.
Circuit Breaker & Fallback	apps/api/src/agents/providers.py:OllamaProvider	Monitors endpoint health, latency, and rate limits. Gracefully degrades to local offline weights or cached fixtures during network outages.
Model Context Protocol (MCP)	apps/api/src/agents/plugins.py	Standardized JSON-RPC plugin architecture enabling dynamic agent capability injection (DOI fact-checkers, local linters, private vector stores).
AST Codebase Ingestion	apps/api/src/agents/document_analyzer.py	Recursively traverses repositories (GitHub API & local filesystem), extracting AST symbol hierarchies and docstrings for technical paper synthesis.

Multi-Agent Swarm Topology & Operational Roles

Agent Designation	Algorithmic Function	Input / Context	Output Artifact
Orchestrator Agent	Decomposes queries into parallel sub-tasks and compiles topological DAG	User Research Query, Target Scope, Source Mode	Execution DAG & Dependency Manifest
Academic Searcher	Queries academic APIs (Semantic Scholar, arXiv) with vector re-ranking	Sub-task hypothesis & keyword vectors	Ranked literature abstracts & peer-reviewed DOIs
Fact-Checker Agent	Verifies mathematical claims, dates, and empirical statements against raw sources	Draft claims + source citations	Citation confidence score & verified DOI list
Document Analyzer	Scans repository trees, architecture configs, and code implementations	GitHub Repo URL or Local directory path	AST architecture map & component specs
Synthesis Lead	Integrates verified claims into LaTeX / Markdown research paper	Verified claims, AST specs, DOI references	Final publication paper & ReportLab PDF

Part I.1: Autonomous Multi-Agent DAG Orchestration & Dynamic Swarm Execution

★ Deterministic Asynchronous Swarm Scheduling — V.A. Sai Venkatesh (Lead Architect)

"Monolithic agents choke on complex research because linear thinking accumulates context noise. In Quorum, research is treated as a directed acyclic graph of mathematical and empirical dependencies. Parallel agents explore orthogonal literature dimensions simultaneously, synchronizing at validation checkpoints before synthesis can begin."

Systemic Problem & Architectural Need:

Standard AI agents execute linearly: if step 2 makes a flawed assumption, all subsequent steps compound the error. Furthermore, linear execution is slow, taking minutes to synthesize multi-domain reports without real-time streaming progress or deterministic failure recovery.

Deep Engineering Specification:

Quorum's OrchestratorAgent translates raw user queries into an execution DAG using Kahn's algorithm for topological sorting. Parallel worker tasks execute concurrently using Python's `asyncio.gather`, bounded by concurrency semaphores to respect downstream API rate limits. Live state transitions are broadcast to clients via WebSocket events.

Operational Procedures & Execution Lifecycle:

- 1. DAG Generation:** The Orchestrator decomposes the query into sub-hypotheses, mapping dependencies (e.g. Literature Retrieval -> Fact-Checking -> Technical Synthesis).
- 2. Concurrent Dispatch:** Independent tasks execute in parallel swarms across configured model providers.
- 3. Topological Barrier:** Synthesis workers remain blocked until all upstream Fact-Checker tasks emit verified verification tokens.
- 4. Real-Time Telemetry:** Clients receive fine-grained execution events (task started, source fetched, claim verified) with sub-100ms latency.

Part I.2: Academic Fact-Checking, DOI Verification & Hallucination Elimination

★ Mathematical Attribution & Citation Truth — V.A. Sai Venkatesh (Lead Architect)

"In academic and medical research, a 95% accurate paper with 5% fabricated citations is completely worthless. Quorum's Fact-Checker acts as a rigorous peer reviewer: every claim must be backed by a verified DOI, checked against Semantic Scholar, and confirmed via CrossRef before entering the canonical report."

Systemic Problem & Architectural Need:

Generative AI models routinely hallucinate plausible-looking scholarly papers, fake author lists, and dead URLs. Researchers and technical leads waste hours chasing non-existent references, leading to legal and technical liability.

Deep Engineering Specification:

Quorum's FactCheckerAgent pairs with the AcademicDOIVerifierPlugin to validate citations. For every extracted claim, the engine queries the CrossRef REST API and Semantic Scholar graph, verifying that the DOI resolves to a published article, matching author names, publication year, and abstract semantics.

Operational Procedures & Execution Lifecycle:

- 1. Claim Extraction:** The agent parses candidate text blocks, isolating empirical assertions, statistical figures, and citations.
- 2. DOI Cross-Validation:** CrossRef and Semantic Scholar APIs are queried via HTTPS with exponential backoff.
- 3. Semantic Alignment Check:** The retrieved paper abstract is cross-referenced with the draft assertion using embedding cosine similarity.
- 4. Attribution Grading:** Claims lacking verifiable scholarly grounding are either amended with genuine citations or flagged with explicit uncertainty tags.

Part I.3: Codebase & Technical Document Analysis Engine (GitHub + Local Air-Gap)

★ Surgical Codebase Ingestion & Architectural Synthesis — V.A. Sai Venkatesh (Lead Architect)

"Software architecture documentation quickly becomes stale because developers hate writing it manually. Quorum's Document Analysis Engine directly connects to GitHub repositories or local code directories, reading package manifests, AST module trees, and configuration files to synthesize publication-ready architectural specifications."

Systemic Problem & Architectural Need:

Enterprises have proprietary codebases containing hundreds of thousands of lines of code. Engineers need instant, publication-grade documentation explaining dependencies, security boundaries, and architectural patterns without leaking IP to external servers.

Deep Engineering Specification:

The DocumentAnalysisAgent utilizes GitHubConnector for remote repositories or recursive file system traversal for local folders. Files are filtered using an intelligent whitelist (.py, .ts, .rs, .go, .md) and token-budgeted using AST chunking. Key architecture manifests (package.json, pyproject.toml, Dockerfile) are synthesized into formal system specs.

Operational Procedures & Execution Lifecycle:

- 1. Source Selection:** User selects Query, GitHub Repository (<https://github.com/org/repo>), or Local Folder.
- 2. Structural Parsing:** The engine traverses directory trees, omitting build artifacts (node_modules, target, .git).
- 3. Component Extraction:** Manifest files are parsed to establish language stacks, framework versions, and API boundaries.
- 4. Specification Synthesis:** Generates comprehensive documentation: Architecture, CS Patterns, REST Endpoints, and Database Models.

Part I.4: Dual AI Inference Toolchain: Cloud Swarms & Local Offline Ollama

★ Air-Gapped Privacy & High-Throughput Cloud Synergy — V.A. Sai Venkatesh (Lead Architect)

"True enterprise resilience requires absolute sovereignty over AI inference. Quorum supports both high-throughput cloud LLMs (Gemini, Claude, GPT-4o) and 100% offline, air-gapped local execution via Ollama. If cloud connectivity fails, the platform seamlessly switches to local weights with zero operational downtime."

Systemic Problem & Architectural Need:

Government agencies, defense contractors, and healthcare organizations cannot send proprietary data to cloud AI providers. Conversely, small research teams need the frontier reasoning power of cloud foundation models without complex infrastructure setup.

Deep Engineering Specification:

The OllamaProvider implements Quorum's abstract AIProvider interface. It interfaces with local Ollama daemons (<http://localhost:11434>) via streaming HTTP, exposing health telemetry, model selection (Llama 3, Mistral, DeepSeek-Coder), and circuit breakers that fall back cleanly if local GPUs are exhausted.

Operational Procedures & Execution Lifecycle:

- 1. Engine Selection:** User toggles between 'Cloud Foundation Models' and 'Local Air-Gapped Ollama' via the UI or API.
- 2. Heartbeat Verification:** The system queries `/api/tags` to ensure the requested local model is pulled and ready.
- 3. Stream Execution:** Inference streams directly into the agent pipeline with low latency and zero external data egress.
- 4. Resilience Fallback:** If a provider encounters rate limits or connection timeouts, the circuit breaker safely redirects execution.

Part I.5: Model Context Protocol (MCP) Extensibility & Dynamic Plugin Registry

★ Open Plugin Ecosystem & Composable Tool Contracts — V.A. Sai Venkatesh (Lead Architect)

"An autonomous platform is only as capable as its tool integrations. By implementing Anthropic's Model Context Protocol (MCP) standard, Quorum transforms from a static application into an open ecosystem where community plugins can inject new data sources, verification engines, and enterprise connectors dynamically."

Systemic Problem & Architectural Need:

Hardcoded tool integrations create technical debt and require complete application re-deployment every time a new API or database connector is added.

Deep Engineering Specification:

Quorum's MCPPluginRegistry manages dynamic tool registration and execution. Tools conform to the MCP JSON-RPC specification, declaring input schemas and capability scopes. Agents dynamically discover registered plugins and call them using secure, parameter-validated sandboxes.

Operational Procedures & Execution Lifecycle:

- 1. Plugin Discovery:** At startup, Quorum scans registered MCP servers and local plugins (e.g. AcademicDOIVerifierPlugin).
- 2. Schema Registration:** Tool parameters and descriptions are registered into the agent prompt contexts.
- 3. Dynamic Invocation:** During DAG execution, agents invoke tools through the unified PluginAgent.execute_tool() interface.
- 4. Result Ingestion:** Tool outputs are sanitized, validated against expected return schemas, and injected into the agent memory.

Part II: Complete REST & Real-Time API Specification

Core Production Endpoints

Method & Path	Subsystem	Description & Operation
POST /api/orchestrate	DAG Swarm	Initiates multi-agent research DAG from query, GitHub repository, or local folder with selected AI engine.
GET /api/reports/{id}	Report Engine	Retrieves full report details including ordered sections, status, and verified citations with DOI links.
GET /api/reports/{id}/pdf	PDF Compiler	Streams publication-grade, pixel-perfect ReportLab PDF with double borders, running headers, and citations.
POST /api/reports/analyze-repo	Codebase Analyzer	Parses GitHub repo or local path to extract architecture specs and generate technical documentation.
GET /api/system/documentation-pdf	Specification System	Returns the official 9-page Quorum Engineering Specification and Architecture Documentation PDF.
GET /health	Liveness Probe	System health check probe returning PostgreSQL, Redis, and AI provider readiness state.

HTTP Status Code & Resilience Protocol

Status Code	Condition & Cause	Platform Resolution & Handling
200 OK	Successful report retrieval, PDF compilation, or health check	Returns structured JSON payload or binary PDF stream with proper Content-Disposition headers.
400 Bad Request	Malformed query, invalid GitHub URL, or missing target source parameters	FastAPI returns Pydantic validation error detailing exact field requirements.
401 / 403 Forbidden	Missing session token for private projects	Evaluators granted auto-session; private reports require valid user authorization.
503 Unavailable	Local Ollama daemon offline or Cloud API rate limit exceeded	Circuit breaker activates; returns descriptive error with fallback options.

Part III: Data Architecture, Deployment & Architectural Sign-off

PostgreSQL & pgvector Knowledge Schema

Model / Table	Key Fields	Description & Canonical Role	Index Strategy
Report	id, query, status, source_type, source_ref	Root entity tracking research workflow, target repository/directory, and execution status	Primary key, project_id index
ReportSection	id, report_id, heading, content, order_index	Individual synthesis sections ordered sequentially by order_index	Indexed on report_id
Source	id, report_id, title, url, snippet, doi	Verified academic citations, GitHub references, and local source file mappings	Indexed on report_id, doi
AuditLog	id, report_id, agent_name, action, payload	Immutable append-only audit trail recording every agent decision and verification event	Indexed on report_id, created_at

Dual Deployment Architecture (Cloud & Air-Gapped)

Option A — Global Cloud Edge (Vercel + Google Cloud Run):

- Frontend hosted on Vercel Edge Network with sub-50ms SSR latency and public evaluator bypass.
- Backend deployed as containerized FastAPI on Google Cloud Run with Neon Serverless PostgreSQL.
- Fully autonomous horizontal scaling with zero-idle cost and SSL termination.

Option B — 100% Offline Air-Gapped Workstation / Kubernetes:

- Containerized via Docker Compose containing API, Next.js web frontend, and local PostgreSQL.
- Interfaces with local Ollama daemon running Llama 3 or DeepSeek-Coder on local workstation GPUs.
- Zero external network packets emitted, ensuring 100% privacy for proprietary repositories and research.

★ ARCHITECTURAL SIGN-OFF — V.A. Sai Venkatesh (Lead Architect)

"Quorum establishes the new benchmark for autonomous AI research and software engineering documentation. By fusing topological DAG swarms, rigorous academic DOI fact-checking, seamless offline Ollama execution, and Model Context Protocol extensibility, the platform eliminates hallucination and guarantees verifiable, publication-grade output."